

COMPLETE SOLVED QUESTION PAPER

Artificial Intelligence / AI & ML – University Examination (7-mark theory answers)

Paper Overview (as read from the two images)

Q.	Part	Topic	Marks
Q1	(ii), (iv)	Top of page is cut off – only "(ii) CSPs" and "(iv) Heuristics" are visible (see note below)	–
Q2	(a)	Define AI; characteristics of an intelligent agent	7*
Q2	(b)	Heuristic search; A* algorithm with example	7*
Q3	(a)	Minimax algorithm; Alpha-Beta pruning with game tree	7
Q3	(b)	CSP definition; Means-End Analysis	7
Q4	(a)	Inference rules + truth table: $P \rightarrow (Q \rightarrow R)$, $P \wedge Q \vdash R$	7
Q4	(b)	Semantic Networks vs Conceptual Dependency: "John gave a book to Mary."	7*
Q5	(a)	Resolution in FOL: "Socrates is mortal"	7
Q5	(b)	Parsing and semantics in formal language processing	7
Q6	(a)	Chomsky classification of 4 grammars	8
Q6	(b)	Parse tree + derivation for $id + id \times id$	6
Q7	(a)	Recursive Transition Network; "The boy eats an apple."	7
Q7	(b)	Utility Theory in AI; expected-utility example	7
Q8	(a)	Game Theory; cooperative vs non-cooperative; Nash; Prisoner's Dilemma	7
Q8	(b)	AI/ML for real-time decisions (traffic / fraud / healthcare / recommendation)	7
Q9	(a)	Passive vs Active RL; exploration vs exploitation; example	7
Q9	(b)	Short note on ANY TWO: Deep RL / CFG / BFS vs DFS	7

- **Reading notes:** Only **Q.6** and **Q.7** labels are clearly printed. The other question numbers (Q2–Q5, Q8, Q9) are **inferred** from sequence – please match them with your original paper.
- Marks marked * are partly cut off at the page edge (the visible "[7" suggests 7).
- The first line "(ii) CSPs ... (iv) Heuristics" belongs to a question whose stem is not visible, so I have **not invented** a question for it. CSPs are fully covered in Q3(b) and heuristics in Q2(b), which are the same topics.
- Because you asked to prepare the full question bank, **every** option of "any two" (Q9b) and every listed domain (Q8b) is answered.

Q2

(a) Define Artificial Intelligence. Explain the characteristics of an intelligent agent with examples.

Answer:

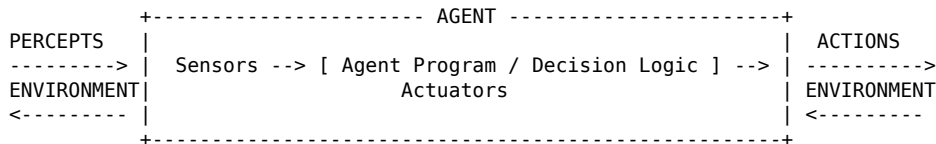
1. Definition

Artificial Intelligence (AI) is the branch of computer science that builds machines/software able to perform tasks that normally need **human intelligence** – such as learning, reasoning, problem-solving, perception, understanding language and decision-making.

Approach	Meaning	Example
Thinking humanly	Model how humans think (cognitive science)	Cognitive models
Acting humanly	Pass the Turing Test	Chatbots
Thinking rationally	Logic-based laws of thought	Theorem provers
Acting rationally	Do the best possible action to achieve goals (rational agent)	Self-driving car

2. Intelligent Agent

An **agent** is anything that **perceives** its environment through **sensors** and **acts** on it through **actuators**. An **intelligent (rational) agent** selects the action expected to maximise its performance measure, given its percept history and knowledge.



3. Characteristics of an Intelligent Agent

Characteristic	Explanation	Example
Autonomy	Works on its own without constant human control; uses own experience	Robot vacuum cleans a room by itself
Perception (Perceptivity)	Senses the environment through sensors	Camera/LiDAR of a self-driving car
Reactivity	Responds in time to changes in the environment	Thermostat turns heater ON when temperature falls
Pro-activeness (Goal-directed)	Takes initiative to achieve goals, not only reacting	Google Maps re-plans route to reach destination early
Rationality	Chooses action that maximises expected performance	Chess engine picks best move
Learning / Adaptability	Improves performance from experience	Spam filter learns new spam patterns
Social ability	Communicates / cooperates with other agents or humans	Siri/Alexa talking to users; multi-robot warehouse
Knowledge / Reasoning	Keeps a knowledge base and infers new facts	Medical expert system
Persistence & Robustness	Keeps running continuously and handles errors/noise	Home-security monitoring agent

4. PEAS description (example: automated taxi)

P - Performance	E - Environment	A - Actuators	S - Sensors
Safety, speed, comfort, profit	Roads, traffic, pedestrians, weather	Steering, accelerator, brake, horn	Camera, GPS, radar, speedometer

5. Types of agents (brief)

- **Simple reflex** – condition-action rules (thermostat).
- **Model-based reflex** – keeps internal state.
- **Goal-based** – chooses actions to reach goals (route planner).
- **Utility-based** – maximises a utility/happiness value.
- **Learning agent** – improves through feedback.

Exam Key Points

- **AI = machines performing tasks needing human intelligence.**
- Agent = sensors + actuators + agent program.
- Rational agent = maximises expected performance measure.
- Memorise: Autonomy, Reactivity, Pro-activeness, Social ability, Learning, Rationality.
- Always give one real example per characteristic.

(b) Describe Heuristic Search. Explain the working of the A* algorithm with a suitable example.

Answer:

1. Heuristic Search - Definition & Purpose

Heuristic search (informed search) uses **problem-specific knowledge** in the form of a **heuristic function $h(n)$** that estimates the cost from node n to the nearest goal. It guides the search toward promising nodes, so far fewer nodes are expanded than in blind (uninformed) search such as BFS/DFS.

Basis	Uninformed search	Heuristic (informed) search
Knowledge	No extra knowledge	Uses $h(n)$
Efficiency	Slow, explores many nodes	Faster, explores fewer nodes
Examples	BFS, DFS, UCS	Greedy Best-First, A*, Hill climbing

Example heuristics: straight-line distance (route finding); number of misplaced tiles / Manhattan distance (8-puzzle).

2. A* Algorithm

A* is a best-first search that expands the node with the **lowest $f(n) = g(n) + h(n)$** .

- **$g(n)$** = actual cost from the start node to n .
- **$h(n)$** = estimated cost from n to the goal (heuristic).
- **$f(n)$** = estimated total cost of the cheapest path through n .

Conditions for optimality: $h(n)$ must be **admissible** (never overestimates: $h(n) \leq \text{true cost } h^*(n)$) and, for graph search, **consistent** ($h(n) \leq c(n, n') + h(n')$).

3. Step-by-step working

- 1 Put the start node in OPEN with $f = g + h$; CLOSED = empty.
- 2 Pick the node n in OPEN with the **smallest $f(n)$** .
- 3 If n is the goal $\rightarrow$ stop, return the path.
- 4 Otherwise move n to CLOSED and generate its successors.
- 5 For each successor: compute $g = g(n) + \text{cost}$; $f = g + h$. If it is new, add to OPEN; if already present with a higher g , update to the cheaper path.
- 6 Repeat from step 2 until the goal is selected or OPEN is empty (failure).

```
A*(start, goal):
    OPEN = {start}; CLOSED = {}
    g[start] = 0; f[start] = h(start)
    while OPEN not empty:
        n = node in OPEN with minimum f
        if n == goal: return path(n)
```

```

move n from OPEN to CLOSED
for each successor m of n:
    new_g = g[n] + cost(n,m)
    if m not in OPEN/CLOSED or new_g < g[m]:
        g[m] = new_g; f[m] = g[m] + h(m); parent[m] = n
        put m in OPEN
return FAILURE

```

4. Example

Graph (edge costs): S-A = 1, S-B = 4, A-B = 2, A-C = 5, A-G = 12, B-C = 2, C-G = 3. Heuristic values: **h(S)=7, h(A)=6, h(B)=4, h(C)=2, h(G)=0** (all admissible).

$\begin{matrix} & 1 & & 2 & & 3 \\ S & \text{-----} & A & \text{-----} & B & \text{-----} & C & \text{-----} & G \end{matrix}$
 (also S-B=4, A-C=5, A-G=12)

Step	Node expanded	Successors generated: g, h, f	OPEN list (f)
1	S (f=7)	A: g=1,h=6, f=7 ; B: g=4,h=4,f=8	A(7), B(8)
2	A (f=7)	B via A: g=3,f=7 (<i>better than 8, update</i>); C: g=6,f=8; G: g=13,f=13	B(7), C(8), G(13)
3	B (f=7)	C via B: g=5,h=2, f=7 (<i>better than 8, update</i>)	C(7), G(13)
4	C (f=7)	G via C: g=8,h=0, f=8 (<i>better than 13, update</i>)	G(8)
5	G (f=8)	Goal selected → STOP	-

Final Answer: Optimal path **S → A → B → C → G** with total cost **1+2+2+3 = 8**. The direct route A→G (cost 13) was never chosen because its f-value was larger.

5. Advantages & Limitations

- **Complete** and **optimal** (with admissible h); optimally efficient among similar algorithms.
- Time and especially **space** can be exponential (stores all nodes) – memory is the main drawback.
- Quality depends on how good h(n) is. If h=0 → Dijkstra/UCS; if g=0 → Greedy best-first.

Exam Key Points

- **f(n) = g(n) + h(n)**; always expand minimum f.
- Admissible: $h(n) \leq \text{actual cost}$ → guarantees optimal path.
- $A^* = \text{UCS (g)} + \text{Greedy (h)}$.
- Draw the table of OPEN list at each step in exam.
- Limitation: high memory use.

Q3

(a) Explain the Minimax algorithm for game playing. How does alpha-beta pruning optimize it? Show with a game tree example.

Answer:

1. Minimax - Definition & Purpose

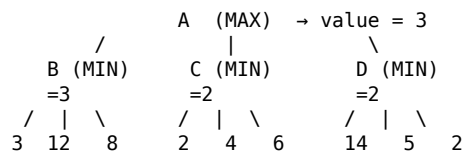
Minimax is a **decision rule for two-player, zero-sum, perfect-information games** (chess, tic-tac-toe). **MAX** tries to maximise the score, **MIN** (the opponent) tries to minimise it. The algorithm assumes the opponent plays **optimally**.

2. Main idea & steps

- 1 Generate the game tree up to a depth limit (or terminal states).
- 2 Apply the **utility/evaluation function** to leaf nodes.
- 3 Back values up: a **MAX** node takes the **maximum** of its children; a **MIN** node takes the **minimum**.
- 4 At the root MAX chooses the move leading to the child with the highest value.

```
MINIMAX(state, depth, player):
    if depth == 0 or terminal(state): return EVAL(state)
    if player == MAX:
        best = -inf
        for each child: best = max(best, MINIMAX(child, depth-1, MIN))
        return best
    else:
        best = +inf
        for each child: best = min(best, MINIMAX(child, depth-1, MAX))
        return best
```

3. Game tree example (MAX root, three MIN children)



Step 1: $B = \min(3, 12, 8) = 3$. **Step 2:** $C = \min(2, 4, 6) = 2$. **Step 3:** $D = \min(14, 5, 2) = 2$. **Step 4:** $A = \max(3, 2, 2) = 3$. **Final Answer:** MAX should choose the move to **B**, guaranteed value 3. Here all 9 leaves are evaluated.

4. Alpha-Beta Pruning

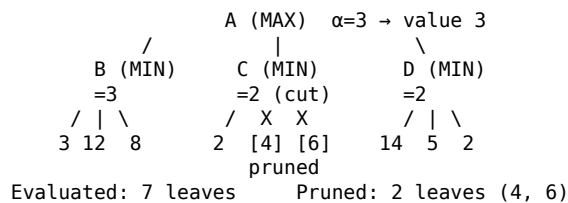
Alpha-beta pruning is an optimisation that gives the **same result as minimax** but **skips (prunes) branches that cannot affect the final decision**.

- **α (alpha)** = best (highest) value MAX can guarantee so far along the path. Initially $-\infty$.
- **β (beta)** = best (lowest) value MIN can guarantee so far along the path. Initially $+\infty$.
- **Pruning rule:** if $\alpha \geq \beta$, stop exploring the remaining children of that node.

```
ALPHABETA(state, depth, alpha, beta, player):
    if depth == 0 or terminal(state): return EVAL(state)
    if player == MAX:
        v = -inf
        for each child:
            v = max(v, ALPHABETA(child, depth-1, alpha, beta, MIN))
            alpha = max(alpha, v)
            if alpha >= beta: break # beta cut-off
        return v
    else:
        v = +inf
        for each child:
            v = min(v, ALPHABETA(child, depth-1, alpha, beta, MAX))
            beta = min(beta, v)
            if alpha >= beta: break # alpha cut-off
        return v
```

5. Alpha-beta on the same tree

Step	Node	Action	α	β	Result
1	A (root)	Start	$-\infty$	$+\infty$	-
2	B	Leaf 3 $\rightarrow \beta=3$; leaf 12 $\rightarrow$ no change; leaf 8 $\rightarrow$ no change	$-\infty$	3	B = 3; back at A: $\alpha = 3$
3	C	Leaf 2 $\rightarrow \beta = 2$. Now $\alpha(3) \geq \beta(2) \rightarrow$ PRUNE leaves 4 and 6	3	2	C = 2 (≤ 2), A stays 3
4	D	Leaf 14 $\rightarrow \beta=14$; leaf 5 $\rightarrow \beta=5$; leaf 2 $\rightarrow \beta=2$ (no prune as it is last leaf)	3	2	D = 2
5	A	$\max(3,2,2)$	3	$+\infty$	A = 3



Basis	Minimax	Alpha-Beta
Nodes examined	All nodes	Fewer (prunes useless branches)
Result	Optimal	Same optimal result
Complexity	$O(b^d)$	Best case $O(b^{d/2})$ - doubles search depth
Move ordering	Not needed	Good ordering $\rightarrow$ more pruning

Exam Key Points

- Minimax: MAX picks max, MIN picks min; assumes optimal opponent.
- **Alpha = best for MAX, Beta = best for MIN; prune when $\alpha \geq \beta$.**
- Alpha-beta gives the **same answer** as minimax with fewer nodes.
- Best-case complexity $O(b^{d/2})$ vs $O(b^d)$.
- Always draw the tree and mark pruned branches with X.

(b) Define a Constraint Satisfaction Problem (CSP). Explain how Means-End Analysis can be used to reduce the difference between current and goal states.

Answer:

1. Constraint Satisfaction Problem (CSP)

A **CSP** is a problem defined by a triple **(X, D, C)**:

- **X** = set of **variables** $\{X_1 \dots X_n\}$
- **D** = **domain** of allowed values for each variable
- **C** = set of **constraints** restricting which value combinations are allowed

A **solution** is a **complete** assignment (every variable has a value) that is **consistent** (violates no constraint). Examples: map colouring, Sudoku, N-Queens, timetable scheduling, cryptarithmic.

Example - Map colouring: Regions A, B, C are all mutually adjacent. Colours = {Red, Green, Blue}. Constraint: $A \neq B$, $B \neq C$, $A \neq C$. One solution: A=Red, B=Green, C=Blue.

Solving techniques: Backtracking search, Forward checking, Arc consistency (AC-3), heuristics MRV (minimum remaining values), degree and least-constraining-value.

2. Means-End Analysis (MEA)

Means-End Analysis (used in the **General Problem Solver, Newell & Simon**) is a problem-solving technique that repeatedly **finds the difference between the current state and the goal state**, then selects an **operator** that **reduces that difference**. It combines forward and backward reasoning.

3. Step-by-step working

- 1 Compare **current state** with **goal state**; if identical → stop.
- 2 Identify the **most important difference**.
- 3 Choose an **operator** that reduces that difference.
- 4 If the operator's **preconditions** are not satisfied, create a **sub-goal** to satisfy them (recursive MEA).
- 5 Apply the operator → new current state. Repeat.

```
MEA(current, goal):  
    if current == goal: return success  
    d = most_important_difference(current, goal)  
    op = operator_that_reduces(d)  
    if preconditions(op) not true in current:  
        MEA(current, preconditions(op))    # sub-goal  
    current = apply(op, current)  
    MEA(current, goal)
```

4. Example - Robot must deliver a box to Room B

Item	Description
Current state	Robot in Room A, box in Room A
Goal state	Box in Room B
Difference	Box location is A, should be B
Operator	CARRY(box, A→B)
Precondition of operator	Robot must be holding the box AND be at A
Sub-goal 1	Robot holds the box → operator PICKUP(box) (precondition: robot at box location A ✓)
Sub-goal 2	Move to B → operator MOVE(A→B)
Final	PICKUP → MOVE → PUTDOWN gives goal state; difference = 0

5. Link with CSP (states, goals, operators, constraints)

MEA element	Map-colouring CSP
State	Partial assignment, e.g. A=Red, B=?, C=?
Goal	All variables assigned with no constraint violated
Difference	Unassigned variables / constraint violations
Operator	Assign(variable, colour) respecting constraints
Working	A=Red (differences: B, C unassigned) → B=Green (one difference left) → C=Blue (difference = 0 ⇒ goal)

Exam Key Points

- **CSP = (X, D, C)**; solution = complete + consistent assignment.
- MEA = find difference → choose operator → satisfy preconditions via sub-goals.
- MEA was the core of Newell-Simon's General Problem Solver (GPS).
- Give a map-colouring example for CSP and robot/box example for MEA.
- Backtracking + forward checking + MRV are standard CSP solvers.

Q4

(a) Given premises: 1. $P \rightarrow (Q \rightarrow R)$ 2. $P \wedge Q$. (i) Derive R step by step. (ii) Verify the validity using a truth table.

Answer:

(i) Derivation using rules of inference

Step	Statement	Justification
1	$P \rightarrow (Q \rightarrow R)$	Premise 1
2	$P \wedge Q$	Premise 2
3	P	Simplification ($\wedge$ -elimination) on 2
4	Q	Simplification on 2
5	$Q \rightarrow R$	Modus Ponens on 1 and 3 ($P, P \rightarrow (Q \rightarrow R) \vdash Q \rightarrow R$)
6	R	Modus Ponens on 5 and 4 ($Q, Q \rightarrow R \vdash R$)

Final Answer: R is derived: $\{P \rightarrow (Q \rightarrow R), P \wedge Q\} \vdash R$.

(ii) Truth-table verification

Argument is **valid** if in every row where **both premises are TRUE**, the conclusion R is also TRUE.

P	Q	R	$Q \rightarrow R$	Premise 1: $P \rightarrow (Q \rightarrow R)$	Premise 2: $P \wedge Q$	Both premises true?	Conclusion R
T	T	T	T	T	T	YES	T ✓
T	T	F	F	F	T	No	F
T	F	T	T	T	F	No	T
T	F	F	T	T	F	No	F
F	T	T	T	T	F	No	T
F	T	F	F	T	F	No	F
F	F	T	T	T	F	No	T
F	F	F	T	T	F	No	F

Observation: Only row 1 ($P=T, Q=T, R=T$) makes both premises true, and in that row $R = \text{TRUE}$. There is **no row** with true premises and false conclusion. Hence the conclusion R is **logically valid** (a tautology: $((P \rightarrow (Q \rightarrow R)) \wedge (P \wedge Q)) \rightarrow R$).

Exam Key Points

- Rules used: **Simplification** and **Modus Ponens** (twice).
- Validity = no row with true premises and false conclusion.
- Only one row (TTT) has both premises true.
- $P \rightarrow (Q \rightarrow R)$ is equivalent to $(P \wedge Q) \rightarrow R$ (exportation law).
- Write each step with its justification.

(b) What are Semantic Networks and Conceptual Dependencies? Represent "John gave a book to Mary" using both and compare. Explain how CD captures meaning better than a semantic net.

Answer:

1. Semantic Network

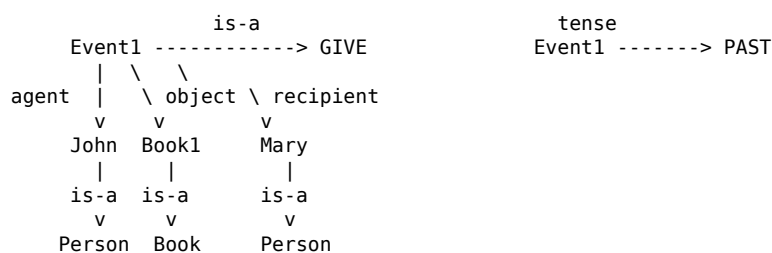
A **semantic network** is a **graph** for representing knowledge: **nodes** = objects/concepts/events, **labelled arcs** = relations (is-a, has-part, agent, object...). It supports **inheritance** through *is-a* links.

2. Conceptual Dependency (CD) - Schank

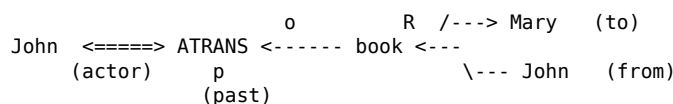
Conceptual Dependency represents the **meaning of a sentence independent of the words used**, using a small set of **primitive acts** and conceptual cases (actor, object, direction, instrument).

Primitive	Meaning	Primitive	Meaning
ATRANS	Transfer of possession/ownership (give, take)	PTRANS	Physical transfer of location (go, move)
MTRANS	Transfer of mental information (tell)	MBUILD	Build new information (decide)
PROPEL	Apply force (push, throw)	MOVE	Move a body part (kick)
GRASP	Grasp an object	INGEST	Take inside body (eat, drink)
EXPEL	Expel from body	SPEAK / ATTEND	Produce sound / focus a sense organ

3. "John gave a book to Mary" - Semantic Network



4. Same sentence - Conceptual Dependency



Written as: ACTOR : John
 ACT : ATRANS (transfer of possession)
 OBJECT : book
 FROM : John TO : Mary
 TENSE : past

Key: ⇔ = actor-action link; **o** = object case; **R** = recipient (direction) case; **p** = past tense.

5. Comparison

Basis	Semantic Network	Conceptual Dependency
Basic idea	Graph of concepts and relations	Meaning built from primitive acts
Primitives	Unlimited relation labels, no fixed set	Small fixed set (ATRANS, PTRANS, MTRANS...)
Word dependence	Depends on the verb used (give, hand, present are different nodes)	Independent of words – paraphrases give same structure
Inference	Mainly inheritance via is-a	Rich inferences from each primitive
Ambiguity	Higher	Lower
Best for	Taxonomies, general knowledge	Understanding events/stories

6. How CD captures meaning better

- **Paraphrase invariance:** "John gave Mary a book", "Mary received a book from John", "John handed Mary the book" all map to **the same ATRANS structure**. A semantic net needs separate verbs/nodes.
- **Built-in inferences:** ATRANS implies (a) Mary **now possesses** the book, (b) John **no longer possesses** it, (c) John probably wanted Mary to have it. These are not explicit in a semantic net.
- **Distinguishes ATRANS from PTRANS:** "gave" = ownership change, not physical movement – CD makes this explicit.
- **Canonical form:** reduces storage and makes question-answering easier (Who has the book now? → Mary).

Exam Key Points

- Semantic net = graph (nodes + labelled arcs); CD = primitive acts + cases.
- "Give" = **ATRANS** with Actor John, Object book, From John, To Mary, past.
- CD gives the **same representation for paraphrases** and supports inference.
- Name primitives: ATRANS, PTRANS, MTRANS, MBUILD, PROPEL, MOVE, GRASP, INGEST, EXPEL, SPEAK, ATTEND.
- Draw both diagrams and a comparison table.

Q5

(a) Describe the Resolution principle in First-Order Predicate Logic. Show how resolution can be used to prove "Socrates is mortal" from: 1. All men are mortal. 2. Socrates is a man.

Answer:

1. Resolution principle - Definition & Purpose

Resolution is a **sound and refutation-complete** inference rule for FOL. It proves a goal by **refutation**: add the **negation of the goal** to the knowledge base (in clause/CNF form) and show that a **contradiction (empty clause $\square$)** follows. Hence the goal must be true.

Resolution rule: from clauses $(L \vee A)$ and $(\neg L' \vee B)$, where L and L' unify with **MGU θ** , infer **$(A \vee B)\theta$** .

2. Main steps of the method

- 1 Convert all premises to FOL.
- 2 Convert to **CNF** (eliminate $\leftrightarrow$ and $\rightarrow$, move $\neg$ inward, standardise variables, Skolemise $\exists$, drop $\forall$, distribute $\vee$ over $\wedge$).
- 3 **Negate the goal** and add its clause.
- 4 Repeatedly select two clauses with complementary literals, **unify** them (substitution), and add the **resolvent**.
- 5 If the **empty clause $\square$** is produced $\rightarrow$ goal is proved.

3. Solution

Step 1 - Predicate representation

- $\text{Man}(x)$: x is a man; $\text{Mortal}(x)$: x is mortal; constant **Socrates**.
- Premise 1: $\forall x [\text{Man}(x) \rightarrow \text{Mortal}(x)]$
- Premise 2: **$\text{Man}(\text{Socrates})$**
- Goal: **$\text{Mortal}(\text{Socrates})$**

Step 2 - Conversion to CNF

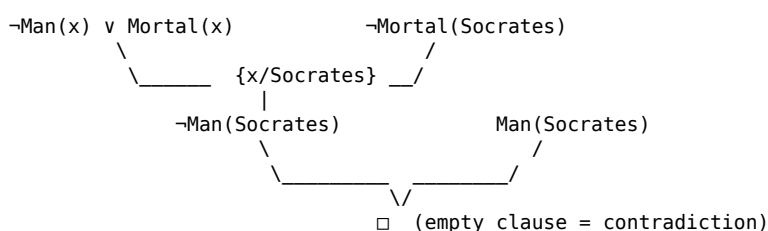
- Eliminate implication: $\forall x [\neg \text{Man}(x) \vee \text{Mortal}(x)]$
- Drop $\forall$ (all variables universally quantified): **C1: $\neg \text{Man}(x) \vee \text{Mortal}(x)$**
- Premise 2 is already a clause: **C2: $\text{Man}(\text{Socrates})$**

Step 3 - Negate the goal

- $\neg \text{Mortal}(\text{Socrates}) \rightarrow$ **C3: $\neg \text{Mortal}(\text{Socrates})$**

Step 4 - Substitution/Unification and Resolution

Step	Clauses resolved	Unifier (MGU)	Resolvent
R1	C1: $\neg \text{Man}(x) \vee \text{Mortal}(x)$ and C3: $\neg \text{Mortal}(\text{Socrates})$	$\theta = \{x / \text{Socrates}\}$ (unify $\text{Mortal}(x)$ with $\text{Mortal}(\text{Socrates})$)	C4: $\neg \text{Man}(\text{Socrates})$
R2	C4: $\neg \text{Man}(\text{Socrates})$ and C2: $\text{Man}(\text{Socrates})$	No substitution needed (identical atoms)	$\square$ (empty clause)



Step 5 - Conclusion: Assuming $\neg \text{Mortal}(\text{Socrates})$ leads to a contradiction, so the assumption is false. Therefore **$\text{Mortal}(\text{Socrates})$ is TRUE - "Socrates is mortal"**.

Exam Key Points

- Resolution = **proof by refutation** (negate goal $\rightarrow$ derive $\square$).
- Unification finds the substitution (MGU) that makes literals identical: $\{x/\text{Socrates}\}$.
- CNF conversion: remove $\rightarrow$, push $\neg$, standardise, Skolemise, drop $\forall$.
- Resolution is **sound** and **refutation-complete**.
- Always show clauses C1–C3 and each resolvent.

(b) Define parsing and semantics in formal language processing. Explain with an example how a parser analyzes the syntactic structure of a sentence and how semantic analysis ensures that the sentence has a valid meaning.

Answer:

1. Definitions

- **Parsing (syntactic analysis)** = analysing a string of tokens/words according to a **grammar** to check it is well-formed and to produce its **structure (parse tree)**.
- **Semantics** = the **meaning** of the sentence. **Semantic analysis** maps the parse tree to a meaning representation and checks that it is **meaningful and consistent**.

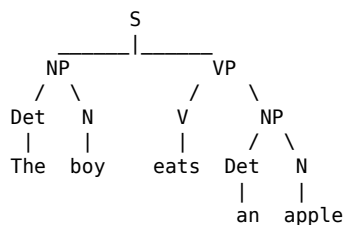
2. Language-processing pipeline

Input text $\rightarrow$ [Lexical analysis] $\rightarrow$ [Syntactic analysis / Parser] $\rightarrow$ [Semantic analysis] $\rightarrow$ [Pragmatics] $\rightarrow$ Meaning
(tokens, POS) (parse tree, grammar rules) (meaning, type checks) (context)

3. How a parser works (example: "The boy eats an apple.")

Grammar: $S \rightarrow NP VP$; $NP \rightarrow Det N$; $VP \rightarrow V NP$; $Det \rightarrow \text{the} \mid \text{an}$; $N \rightarrow \text{boy} \mid \text{apple}$; $V \rightarrow \text{eats}$.

- **Top-down parsing:** start with S and expand rules to match the words.
- **Bottom-up parsing:** start with words, reduce them to non-terminals up to S.



Derivation (leftmost): $S \Rightarrow NP VP \Rightarrow Det N VP \Rightarrow \text{the } N VP \Rightarrow \text{the boy } VP \Rightarrow \text{the boy } V NP \Rightarrow \text{the boy eats } NP \Rightarrow \text{the boy eats } Det N \Rightarrow \text{the boy eats an } N \Rightarrow \text{the boy eats an apple}$. The parser accepts: the sentence is **syntactically valid**.

4. Semantic analysis ensures valid meaning

- **Meaning representation (FOL):** $\exists x [\text{Apple}(x) \wedge \text{Eats}(\text{boy}, x)]$.
- **Selectional restrictions:** the verb “eat” needs an **animate agent** and an **edible object**. boy = animate $\checkmark$; apple = edible $\checkmark \rightarrow$ **valid meaning**.
- **Counter-example:** “The apple eats the boy.” – syntax is perfect (same $S \rightarrow NP VP$), but semantics fails (apple is not an animate eater) $\rightarrow$ rejected. Also, “Colorless green ideas sleep furiously” is grammatical but meaningless.
- Semantic analysis also resolves **word-sense ambiguity** (bank = river bank / money bank), checks agreement and types.

Basis	Parsing (syntax)	Semantic analysis
Concerns	Structure / grammar	Meaning
Output	Parse tree	Logical form / meaning representation
Detects	Grammar errors	Meaningless or inconsistent sentences

Basis	Parsing (syntax)	Semantic analysis
Example	"The apple eats the boy" passes	"The apple eats the boy" fails

Exam Key Points

- Parsing = structure by grammar → parse tree; semantics = meaning.
- Parsers: top-down and bottom-up.
- Syntactically correct ≠ semantically valid.
- Selectional restrictions (animate agent, edible object) check meaning.
- Use the "The boy eats an apple" tree and the "apple eats boy" counter-example.

Q6

(a) Classify the following grammars according to Chomsky's hierarchy, giving justification for each: (i) $S \rightarrow aSb \mid ab$ (ii) $S \rightarrow aA, A \rightarrow aAb \mid \epsilon$ (iii) $S \rightarrow aSbS \mid \epsilon$ (iv) $S \rightarrow aSb \mid bSa \mid \epsilon$

Answer:

1. Chomsky hierarchy (reference)

Type	Grammar	Production form	Machine	Example language
Type 0	Unrestricted	$\alpha \rightarrow \beta$ (α contains at least one non-terminal)	Turing machine	Any computable language
Type 1	Context-sensitive	$\alpha A \beta \rightarrow \alpha \gamma \beta, LHS \leq RHS $	Linear-bounded automaton	$a^n b^n c^n$
Type 2	Context-free	$A \rightarrow \gamma$ (single non-terminal on the left)	Pushdown automaton	$a^n b^n$
Type 3	Regular	$A \rightarrow aB$ or $A \rightarrow a$ (right-linear)	Finite automaton	$a^* b^*$

Test used: (1) Is every rule $A \rightarrow a$ or $A \rightarrow aB$ (one terminal, at most one non-terminal at the right end)? $\rightarrow$ Type 3. (2) Is the left side always a **single non-terminal**? $\rightarrow$ Type 2. Otherwise check Type 1 / Type 0. Grammar type = the **most restrictive** type satisfied.

(i) $S \rightarrow aSb \mid ab$

- LHS is a single non-terminal in every rule $\rightarrow$ **Type 2 (Context-Free)**.
- $S \rightarrow aSb$ has a non-terminal in the **middle** (not at the right end) $\rightarrow$ not right-linear $\rightarrow$ **not Type 3**.
- Language: $L = \{a^n b^n \mid n \geq 1\}$. Equal counting of a's and b's cannot be done by a finite automaton, so it is not regular.
- Example: $S \Rightarrow aSb \Rightarrow aabb$.

(ii) $S \rightarrow aA, A \rightarrow aAb \mid \epsilon$

- All LHS are single non-terminals $\rightarrow$ **Type 2 (Context-Free)**.
- $A \rightarrow aAb$ has the non-terminal in the middle $\rightarrow$ not regular. ϵ -production is allowed in CFG.
- A generates $a^n b^n$ ($n \geq 0$); $S \rightarrow aA$ prefixes one extra "a". Language: $L = \{a^{n+1} b^n \mid n \geq 0\}$ (= a, aab, aaabb, ...).
- Example: $S \Rightarrow aA \Rightarrow aaAb \Rightarrow aab$.

(iii) $S \rightarrow aSbS \mid \epsilon$

- Single non-terminal on LHS $\rightarrow$ **Type 2 (Context-Free)**.
- Two occurrences of S on the right side and a non-terminal in the middle $\rightarrow$ not linear $\rightarrow$ not regular.
- Language: **balanced strings** (treat a as "(" and b as ")"): $\epsilon, ab, aabb, abab, \dots$ i.e., every prefix has $\#a \geq \#b$ and total $\#a = \#b$ (Dyck language).
- Example: $S \Rightarrow aSbS \Rightarrow a(aSbS)bS \Rightarrow aabb$.

(iv) $S \rightarrow aSb \mid bSa \mid \epsilon$

- Single non-terminal on LHS $\rightarrow$ **Type 2 (Context-Free)**.
- Recursion is in the middle (S between terminals) $\rightarrow$ not right-linear $\rightarrow$ not Type 3.
- Language: even-length strings in which the **i-th symbol from the left is different from the i-th symbol from the right** (nested pairs ab or ba): $\epsilon, ab, ba, aabb, abab, baab, \dots$ (always equal a's and b's).
- Example: $S \Rightarrow aSb \Rightarrow a(bSa)b \Rightarrow abab$.

Grammar	Type	Language	Reason
(i)	2 - CFG	$a^n b^n, n \geq 1$	Single NT on left; middle recursion

Grammar	Type	Language	Reason
(ii)	2 - CFG	$a^{n+1}b^n, n \geq 0$	Single NT on left; middle recursion
(iii)	2 - CFG	Balanced a/b (Dyck)	Single NT on left; S appears twice
(iv)	2 - CFG	Opposite-symbol palindromic pairs	Single NT on left; middle recursion

Exam Key Points

- All four grammars are **Type 2 (Context-Free)**; none is regular.
- Justification: **LHS = single non-terminal**; non-terminal in the middle of RHS (needs a stack).
- Regular (Type 3) needs $A \rightarrow aB$ / $A \rightarrow a$ only.
- ϵ -productions are allowed in CFGs.
- Always give the language generated and a sample derivation.

(b) Grammar: $E \rightarrow E+T \mid T$; $T \rightarrow T \times F \mid F$; $F \rightarrow (E) \mid \text{id}$. Construct the parse tree and show the derivation for $\text{id} + \text{id} \times \text{id}$.

Answer:

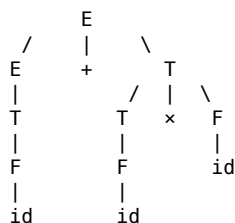
Step 1 - Identify structure: E handles "+", T handles "x". Because x is generated deeper (inside T), it is evaluated **before** + $\rightarrow$ higher precedence.

Step 2 - Leftmost derivation (always expand the leftmost non-terminal):

Step	Sentential form	Rule applied
1	E	start symbol
2	E + T	$E \rightarrow E + T$
3	T + T	$E \rightarrow T$
4	F + T	$T \rightarrow F$
5	id + T	$F \rightarrow \text{id}$
6	id + T $\times$ F	$T \rightarrow T \times F$
7	id + F $\times$ F	$T \rightarrow F$
8	id + id $\times$ F	$F \rightarrow \text{id}$
9	id + id $\times$ id	$F \rightarrow \text{id}$

Rightmost derivation: $E \Rightarrow E+T \Rightarrow E+T \times F \Rightarrow E+T \times \text{id} \Rightarrow E+F \times \text{id} \Rightarrow E+\text{id} \times \text{id} \Rightarrow T+\text{id} \times \text{id} \Rightarrow F+\text{id} \times \text{id} \Rightarrow \text{id}+\text{id} \times \text{id}$.

Step 3 - Parse tree:



Step 4 - Operator precedence and associativity:

- Reading the tree: the root splits as **E + T**, and the right T is **id $\times$ id**. So the string is parsed as **id + (id $\times$ id)**.
- **Precedence:** $\times$ (lower in the grammar hierarchy, nearer the leaves) $>$ + .
- **Associativity:** Both rules are left-recursive ($E \rightarrow E+T$, $T \rightarrow T \times F$) $\Rightarrow$ **left associative**.
- The grammar is **unambiguous**: precedence is built in by using separate levels E, T, F. Parentheses $F \rightarrow (E)$ override precedence.

Final Answer: the string is accepted; it has a single parse tree meaning $\text{id} + (\text{id} \times \text{id})$.

Exam Key Points

- Derivation: $E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow id+T \Rightarrow id+T \times F \Rightarrow id+F \times F \Rightarrow id+id \times F \Rightarrow id+id \times id$.
- $\times$ binds tighter than $+$ (deeper in the tree).
- Left recursion $\rightarrow$ left associativity.
- Levels $E (+)$, $T (\times)$, F (atoms) enforce precedence $\rightarrow$ unambiguous grammar.
- Draw tree neatly with all $F \rightarrow id$ leaves.

Exam Key Points

- **RTN = FSM + arcs that call other networks (recursion) + stack.**
- Equivalent in power to **context-free grammars**.
- Draw three networks: S (NP→VP), NP (Det→Noun), VP (Verb→NP).
- Push on call, pop on return at a final state.
- Needed for nested structures (PP, relative clauses).

(b) Explain Utility Theory in AI. How is utility used to model an agent's preferences and rational decision-making under uncertainty? Illustrate with an example of an AI agent choosing between risky actions based on expected utility.

Answer:

1. Utility and Utility Theory

Utility is a **numerical measure of the desirability (satisfaction/value)** of a state or outcome for an agent. **Utility theory** says an agent's preferences can be represented by a **utility function $U(s)$** , and a **rational agent** chooses actions that **maximise expected utility**. It is the foundation of decision-theoretic and utility-based agents.

2. Preferences

- $A \succ B$: agent prefers A to B; $A \sim B$: indifferent; $A \succeq B$: prefers or indifferent.
- $U(A) > U(B) \Leftrightarrow A \succ B$ - utility turns qualitative preferences into numbers.
- **Axioms of rational preference: Orderability** ($A \succ B$, $B \succ A$ or $A \sim B$), **Transitivity** ($A \succ B$, $B \succ C \Rightarrow A \succ C$), **Continuity**, **Substitutability**, **Monotonicity**, **Decomposability**. If an agent obeys them, a utility function exists.

3. Decision-making under uncertainty

Each action a may lead to several outcomes s' with probabilities $P(s' | a)$. The **Expected Utility** is:

$$EU(a) = \sum_{s'} P(s' | a) \times U(s')$$

Maximum Expected Utility (MEU) principle: $a^* = \operatorname{argmax}_a EU(a)$ - a rational agent selects the action with highest EU.

Risk attitudes: concave U (e.g. $\sqrt{\text{money}}$) → **risk-averse**; linear → **risk-neutral**; convex → **risk-seeking**. This is why utility is not the same as money.

4. Numerical example - Delivery drone choosing a route

Utility scale 0-100 (time saved, safety, battery). Three actions:

Action	Outcome	Probability	Utility	Contribution $P \times U$
A: Safe route (low-altitude, long)	Delivered slowly	1.0	60	60
	EU(A)			60
B: Fast route (city centre)	Delivered quickly	0.6	100	60
	Delayed by wind/obstacles	0.4	20	8
	EU(B)			68
C: Shortcut (over storm area)	Very fast delivery	0.5	120	60
	Crash / package lost	0.5	-20	-10
	EU(C)			50

Final Answer: $EU(A)=60$, $EU(B)=68$, $EU(C)=50$ → by MEU the drone should choose **Action B (fast route)** even though it is risky, since its expected utility is the highest. Note C has the highest best-case (120) but the lowest EU - a rational agent does not choose it. If the drone were strongly **risk-averse** (heavy penalty on crash), U values would change and A might win.

Exam Key Points

- **Utility = numeric measure of desirability;** preferences $\Rightarrow U(A) > U(B)$.
- **$EU(a) = \sum P(s'|a) \cdot U(s')$** ; choose **argmax EU** (MEU).
- Rationality needs the preference axioms (transitivity etc.).
- Risk-averse = concave utility; utility $\neq$ money.
- Write a probability table with EU for each action.

Q8

(a) Explain the concept of Game Theory in AI. Differentiate between cooperative and non-cooperative games, and describe how the Nash Equilibrium is used to find stable strategies. Illustrate with a payoff matrix example for the Prisoner's Dilemma.

Answer:

1. Game theory - definition

Game theory is the mathematical study of **strategic decision-making** where the outcome for each agent depends on the **choices of other agents** too. In AI it is used in **multi-agent systems**, auctions, negotiation, self-driving cars at intersections, security games, and adversarial training (GANs). Elements: **players, strategies/actions, payoffs** (utilities) and **rules**.

2. Cooperative vs non-cooperative games

Basis	Cooperative game	Non-cooperative game
Binding agreements	Players can form binding agreements/coalitions	No binding agreements; each plays for self-interest
Focus	Coalition formation and how to share the gain	Each player's best strategy
Solution concept	Core, Shapley value	Nash equilibrium
Example	Companies forming a joint venture; robots sharing a task	Prisoner's Dilemma, chess, price war
Payoff	Joint payoff then divided	Individual payoffs

3. Nash Equilibrium (NE)

A set of strategies, one per player, is a **Nash equilibrium** if **no player can improve their payoff by changing only their own strategy** while others keep theirs fixed. Each strategy is a **best response** to the others - hence **stable**.

- Step 1: For each strategy of the opponent, find your **best response**.
- Step 2: Mark best-response payoffs for both players.
- Step 3: Cell where **both** are best responses = Nash equilibrium.
- A **dominant strategy** (best whatever the opponent does) always leads to an NE.

4. Prisoner's Dilemma - payoff matrix

Two suspects are interrogated separately. Silent = **Cooperate (C)**, Confess = **Defect (D)**. Payoffs = (years in prison, shown as negative utility) (Prisoner 1, Prisoner 2):

	Prisoner 2: Silent (C)	Prisoner 2: Confess (D)
Prisoner 1: Silent (C)	(-1, -1)	(-3, 0)
Prisoner 1: Confess (D)	(0, -3)	(-2, -2) ← Nash

Step 1 - Best response of Prisoner 1: If P2 is Silent: C gives -1, D gives 0 → **D** better. If P2 Confesses: C gives -3, D gives -2 → **D** better. So **Confess is dominant** for P1.

Step 2 - Best response of Prisoner 2: by symmetry, **Confess** is dominant.

Step 3 - Nash equilibrium: (Confess, Confess) with payoff (-2, -2). Neither can gain by switching alone (switching to Silent gives -3).

Step 4 - The dilemma: (Silent, Silent) = (-1, -1) is **better for both** (Pareto-optimal) but is **not stable** because each is tempted to defect. Individual rationality leads to a collectively worse result. In repeated games, strategies like **Tit-for-Tat** can sustain cooperation.

Exam Key Points

- Game theory = strategic interaction among rational agents.
- Cooperative = binding agreements, share payoff; Non-cooperative = individual strategy, Nash equilibrium.
- **Nash equilibrium: no player gains by unilaterally changing strategy.**
- PD: NE = (Confess, Confess) although (Silent, Silent) is better for both.
- Draw the 2x2 payoff matrix and show best responses.

(b) Discuss how AI and Machine Learning can provide solutions to real-time decision-making problems. Explain with a case study from: (i) Smart traffic management (ii) Fraud detection (iii) Healthcare diagnostics (iv) Personalized recommendation systems

Answer:

General framework - real-time AI decision loop

Data capture (sensors, logs) → Pre-processing/features → ML model (inference in ms)
→ Decision/action (signal, block, alert, recommend) → Feedback → Model retraining

AI/ML helps because it can **process streaming data instantly, learn patterns, predict, and act automatically** - faster and more consistently than humans.

(i) Smart traffic management - case study: adaptive traffic signals

- **Problem:** Fixed-timer signals cause congestion and long waits.
- **Data:** road cameras, loop detectors, GPS from vehicles, emergency-vehicle alerts.
- **Workflow:** sensors count vehicles → ML (computer vision + **reinforcement learning**) predicts queue length → agent chooses signal phase/duration (state = queue lengths, action = green phase, reward = -waiting time) → signals update every few seconds.
- **Example:** Pittsburgh's Surtrac adaptive signals reported large cuts in travel time and waiting; similar systems used in smart-city projects.
- **Benefits:** less congestion, lower fuel use/pollution, priority to ambulances.
- **Challenges:** sensor failure, high-quality real-time data, privacy of camera data, integration with old infrastructure, safety/failure handling.

(ii) Fraud detection - case study: credit-card transactions

- **Problem:** Detect fraudulent transactions within milliseconds, before approval.
- **Data:** amount, merchant, location, time, device, past behaviour.
- **Workflow:** transaction arrives → features (e.g. distance from last transaction, spending deviation) → model (**Random Forest / XGBoost / anomaly detection / neural net**) gives fraud probability → if score > threshold: block, or request OTP → analyst feedback retrains the model.
- **Example:** A card used in Delhi at 2 pm is suddenly used for a large online purchase abroad at 2:05 pm → high anomaly score → transaction declined and customer alerted.
- **Benefits:** reduces financial loss, protects customers, works 24x7.
- **Challenges:** extreme **class imbalance** (fraud is rare), false positives annoy customers, fraudsters adapt (concept drift), explainability.

(iii) Healthcare diagnostics - case study: diabetic retinopathy / chest X-ray

- **Problem:** Early, accurate diagnosis with shortage of specialists.
- **Data:** medical images, lab reports, patient records, ICU vital signs.
- **Workflow:** image/vitals captured → CNN model classifies (e.g. normal / disease grade) in seconds → risk score and highlighted region shown to doctor → doctor confirms → outcome stored for retraining. In ICUs, models on streaming vitals give **sepsis early warnings**.
- **Example:** An AI system screens retinal images for diabetic retinopathy at primary-care clinics and instantly refers high-risk patients.

- **Benefits:** faster diagnosis, wider access, consistent accuracy, early alerts.
- **Challenges:** patient privacy, biased/limited data, need for explainability and doctor oversight, regulatory approval, errors can be life-critical.

(iv) Personalized recommendation systems - case study: Netflix / Amazon / YouTube

- **Problem:** Show each user the most relevant item among millions in real time.
- **Data:** watch/purchase history, clicks, ratings, time of day, device.
- **Workflow:** user opens app → candidate generation (collaborative filtering / matrix factorisation / deep learning) → ranking model scores items → top-N list displayed → clicks/watch-time fed back (online learning; bandits balance exploration-exploitation).
- **Example:** After watching sci-fi movies, the home page ranks similar movies first; Amazon shows “customers who bought this also bought...”.
- **Benefits:** higher engagement and sales, better user experience.
- **Challenges:** cold-start problem, filter bubbles, privacy, bias, scalability and latency.

Exam Key Points

- Real-time AI loop: data → model → decision → feedback.
- Traffic: RL/vision adaptive signals; Fraud: anomaly detection in milliseconds.
- Healthcare: CNN diagnosis + doctor in the loop; Recommenders: collaborative filtering/deep ranking.
- For any case: Problem – Data – Workflow – Example – Benefits – Challenges.
- Common challenges: data quality, privacy, bias, latency, explainability.

Q9

(a) Differentiate between Passive RL and Active RL. Explain how exploration and exploitation play a role in Active RL. Use an example (e.g. robot navigation or game playing) to show how an agent updates both its policy and utility estimates.

Answer:

1. Reinforcement Learning (RL) - basics

In **RL** an agent interacts with an environment, receives **rewards**, and learns to maximise **long-term cumulative reward**. **Utility (value) of a state** $U(s)$ = expected sum of discounted future rewards; a **policy** $\pi(s)$ tells which action to take in state s .

2. Passive vs Active RL

Basis	Passive RL	Active RL
Policy	Fixed policy π given	Agent must learn/decide the policy itself
Goal	Evaluate the policy: learn $U^\pi(s)$	Find the optimal policy π^*
What it learns	Utilities of states under π (and model)	Optimal utilities / Q-values and best actions
Exploration	Not needed – actions are dictated	Essential – must try actions to discover better ones
Methods	Direct utility estimation, ADP, TD learning	Q-learning , SARSA, active ADP
Analogy	Watching someone play and judging how good their strategy is	Playing yourself to find the best strategy
Update rule	$U(s) \leftarrow U(s) + \alpha[r + \gamma U(s') - U(s)]$	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

3. Exploration vs Exploitation in Active RL

- **Exploitation** = choose the action currently believed best (maximise reward now).
- **Exploration** = try less-known actions to gather information (maybe higher reward later).
- **Dilemma**: only exploiting may get stuck in a sub-optimal policy; only exploring wastes reward. A balance is needed.
- **ϵ -greedy**: with probability $1-\epsilon$ exploit (best action), with probability ϵ explore (random action); ϵ decreases over time. Other methods: softmax, UCB, optimistic initial values.

4. Example - Robot navigation (grid) with Q-learning

Robot at state **S1**; actions Up, Right. Goal state S2 (reward +10 when reached). Parameters: learning rate $\alpha = 0.5$, discount $\gamma = 0.9$, $\epsilon = 0.1$. Step cost $r = -1$ per move. Current estimates: $Q(S1, Up) = 2$, $Q(S1, Right) = 0$, and $\max_a Q(S2, a') = 10$ (so $U(S2) = 10$, $U(S1) = 0$).

Step 1 - Select action (ϵ -greedy): With 10% chance explore; here a random draw chooses to **explore** and the robot tries **Right** (exploitation would have chosen Up since $2 > 0$).

Step 2 - Observe: reward $r = -1$, new state S2.

Step 3 - Update Q-value (action-utility):

$$Q(S1, Right) = 0 + 0.5 \times [-1 + 0.9 \times 10 - 0] \\ = 0 + 0.5 \times 8 = 4$$

Step 4 - Update state utility (TD): $U(S1) = 0 + 0.5 \times [-1 + 0.9 \times 10 - 0] = 4$ (or $U(S1) = \max_a Q(S1, a)$).

Step 5 - Update policy: $\pi(S1) = \arg\max_a Q(S1, a)$. Now $Q(S1, Right) = 4 > Q(S1, Up) = 2 \rightarrow$ **policy changes from Up to Right**. Exploration discovered a better action; later exploitation will pick Right.

Final Answer: After one step, $Q(S1, Right) = 4$, $U(S1) = 4$ and $\pi(S1) = \text{Right}$. Repeating many episodes, Q-values converge to the optimal values and policy π^* .

Game-playing example: A game agent (e.g. TD-Gammon / AlphaGo) tries new moves (explore) and increasingly plays moves with the highest learned value (exploit).

Exam Key Points

- **Passive = fixed policy, learn utilities; Active = learn optimal policy.**
- Active RL needs exploration; use ϵ -greedy for balance.
- TD: $U(s) \leftarrow U(s) + \alpha[r + \gamma U(s') - U(s)]$; Q-learning: $Q \leftarrow Q + \alpha[r + \gamma \max_{a'} Q(s', a) - Q]$.
- Policy = argmax of Q; updates when a better action is found.
- Give a small numeric update like above.

(b) Write short notes on any two: (i) Deep Reinforcement Learning (ii) Context-Free Grammar (iii) Breadth-first search vs Depth-first search

(Question says "any two"; all three are answered for the question bank. Write any two in the exam.)

(i) Deep Reinforcement Learning (Deep RL)

Deep RL combines **reinforcement learning** with **deep neural networks**. The network approximates the **value function, Q-function or policy**, so the agent can learn from **high-dimensional inputs** (images, sensor data) where tables are impossible.

State (e.g. game pixels) $\rightarrow$ [Deep Neural Network] $\rightarrow$ $Q(s, a_1), Q(s, a_2), \dots \rightarrow$ choose action
↑
reward, next state $\leftarrow$ Environment

- **Deep Q-Network (DQN):** uses a CNN to estimate Q-values; stabilised by **experience replay** (random mini-batches of past transitions) and a **target network**.
- **Policy-gradient / Actor-Critic** methods (A3C, PPO) learn the policy directly; **AlphaGo / AlphaZero** combine Deep RL with tree search.
- **Applications:** Atari/Go/chess, robotics, self-driving cars, resource management, RLHF for language models.
- **Challenges:** huge data needs, unstable training, reward design, sparse rewards, safety.

(ii) Context-Free Grammar (CFG)

A **CFG** is a **Type-2 grammar** defined by **$G = (V, \Sigma, P, S)$** : V = variables (non-terminals), Σ = terminals, P = productions of the form **$A \rightarrow \alpha$** ($A \in V, \alpha \in (V \cup \Sigma)^*$), S = start symbol.

- **Language:** $L(G)$ = set of all terminal strings derivable from S . Recognised by a **pushdown automaton**.
- **Derivation:** leftmost / rightmost; shown by a **parse (derivation) tree**.
- **Ambiguity:** a grammar is ambiguous if some string has two or more parse trees.
- **Example:** $S \rightarrow aSb \mid \epsilon$ generates $\{a^n b^n \mid n \geq 0\}$; $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$.
- **Uses:** programming-language syntax (compilers), natural-language parsing, XML/HTML structure.
- **Normal forms:** Chomsky Normal Form ($A \rightarrow BC \mid a$), Greibach Normal Form.

(iii) Breadth-First Search (BFS) vs Depth-First Search (DFS)

Basis	BFS	DFS
Strategy	Expands nodes level by level (shallowest first)	Goes as deep as possible, then backtracks
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Completeness	Complete (finite branching)	Not complete in infinite/cyclic graphs (without limit)
Optimality	Optimal if all step costs equal (shortest path)	Not optimal
Time	$O(b^d)$	$O(b^m)$
Space	$O(b^d)$ - high memory	$O(b \cdot m)$ - low memory
Best for	Shortest path, shallow goals	Deep solutions, low memory, puzzles/maze, topological sort
Example order ($A \rightarrow B, C$; $B \rightarrow D, E$)	A, B, C, D, E	A, B, D, E, C

(b = branching factor, d = depth of shallowest goal, m = maximum depth.)

Exam Key Points

- **Deep RL = RL + deep neural network** (DQN: replay + target network).
- **CFG = (V, Σ , P, S), rules $A \rightarrow \alpha$; Type-2; PDA.**
- **BFS = queue, shortest path, high memory; DFS = stack, low memory, may not find shortest.**
- Give one example per note ($a^n b^n$ for CFG, AlphaGo/Atari for Deep RL).
- Write only two in exam as asked.

QUICK REVISION SHEET

Important Definitions

- **AI:** machines performing tasks needing human intelligence.
- **Rational agent:** acts to maximise expected performance measure.
- **Heuristic:** estimate of cost to goal; **admissible** if never overestimates.
- **CSP:** (X, D, C) – variables, domains, constraints.
- **MEA:** reduce the difference between current and goal state using operators.
- **Resolution:** refutation proof – negate goal, derive empty clause.
- **RTN:** FSM whose arcs can call networks (recursion + stack).
- **Utility:** numeric desirability of an outcome. **Nash equilibrium:** no player gains by unilateral change.
- **Passive RL:** evaluate fixed policy; **Active RL:** learn the best policy.

Important Formulas

Topic	Formula
A*	$f(n) = g(n) + h(n)$
Alpha-beta pruning	Prune when $\alpha \geq \beta$
Expected utility	$EU(a) = \sum P(s' a) U(s')$; $a^* = \text{argmax } EU$
TD learning (passive)	$U(s) \leftarrow U(s) + \alpha[r + \gamma U(s') - U(s)]$
Q-learning (active)	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
Resolution rule	$(L \vee A), (\neg L' \vee B), \theta = \text{MGU} \vdash (A \vee B)\theta$
Modus Ponens	$P, P \rightarrow Q \vdash Q$
Exportation	$P \rightarrow (Q \rightarrow R) \equiv (P \wedge Q) \rightarrow R$

Important Algorithms

- A* (OPEN list by f)
- Minimax and Alpha-Beta
- Backtracking / forward checking for CSP; Means-End Analysis
- Resolution with unification (CNF conversion)
- Top-down / bottom-up parsing; RTN parsing
- Q-learning with ϵ -greedy; BFS and DFS

Important Differences / Comparisons

- Uninformed vs heuristic search
- Minimax vs Alpha-Beta
- Semantic Network vs Conceptual Dependency
- Syntax (parsing) vs Semantics
- FSM vs RTN (regular vs context-free)
- Cooperative vs non-cooperative games
- Passive vs Active RL; Exploration vs Exploitation
- BFS vs DFS

Important Diagrams to Practise

- Agent-environment diagram
- A* trace table (OPEN list)
- Minimax tree with pruned branches marked X

- Semantic net and CD (ATRANS) diagram for “John gave a book to Mary”
- Resolution refutation tree for Socrates
- Parse tree for $id + id \times id$
- RTN for “The boy eats an apple” (S, NP, VP networks)
- Prisoner’s Dilemma payoff matrix

Important Examples

Concept	Example
Agent	Self-driving car, thermostat, robot vacuum
A*	Route-finding with straight-line distance
CSP	Map colouring / Sudoku
Chomsky	$a^n b^n$ is Type 2; $a^* b^*$ is Type 3
Utility	Drone route choice: EU 60 / 68 / 50
Game theory	Prisoner’s Dilemma → (Confess, Confess)
RL	Robot navigation Q-update; AlphaGo / Atari for Deep RL

Frequently Repeated Concepts in this Paper

- **Language and grammar theme:** Chomsky classes, CFG, parse trees, derivations, parsing, RTN (Q5b, Q6, Q7a, Q9b).
- **Logic theme:** inference rules, truth tables, resolution, CNF, unification (Q4a, Q5a).
- **Decision/Rational-agent theme:** agent, A*, minimax, utility, game theory, RL (Q2, Q3, Q7b, Q8a, Q9a).
- Comparison / “differentiate” questions appear in many parts (Q4b, Q8a, Q9a, Q9b).

MOST IMPORTANT EXAM TOPICS

Topics covered by the uploaded paper, grouped by area (not ranked).

Group	Topics covered
1. AI fundamentals	Definition of AI, intelligent agents and their characteristics, PEAS, agent types
2. Search and game playing	Heuristic search, A*, Minimax, Alpha-Beta pruning, CSP, Means-End Analysis, BFS vs DFS
3. Logic and knowledge representation	Rules of inference, truth tables, FOL, CNF, unification, resolution, Semantic Networks, Conceptual Dependency
4. Formal languages and grammars	Chomsky hierarchy, Context-Free Grammar, derivations and parse trees (expression grammar)
5. Parsing and NLP	Parsing vs semantic analysis, Recursive Transition Networks
6. Reinforcement learning	Passive vs Active RL, exploration vs exploitation, utility/policy updates, Deep RL
7. Game theory	Cooperative vs non-cooperative games, Nash equilibrium, Prisoner's Dilemma
8. AI/ML applications	Real-time decision making: smart traffic, fraud detection, healthcare diagnostics, recommendation systems
9. Utility and decision-making	Utility theory, preferences, rational decisions, expected utility, risky actions